



The Need for Speed: Cut Your Spring Boot Build Times by 70%

The hidden gem of Spring Test to unlock blazing fast feedback

JavaCro 2026 · October 13, 2026

Philip Riecks · @rieckpil · PragmaTech GmbH

Participate During the Talk

Go to menti.com and enter the code **0000 0000**.

About Philip

- Software Engineer from Herzogenaurach (HQ of adidas & Puma), Germany 🍻
- Blogging and content creation about testing Java and Spring Boot applications 🍃
- Founder of PragmaTech GmbH - **Enabling Developers to Frequently Deliver** Software with **More Confidence**



Herzogenaurach, Bavaria

Why Test Software?

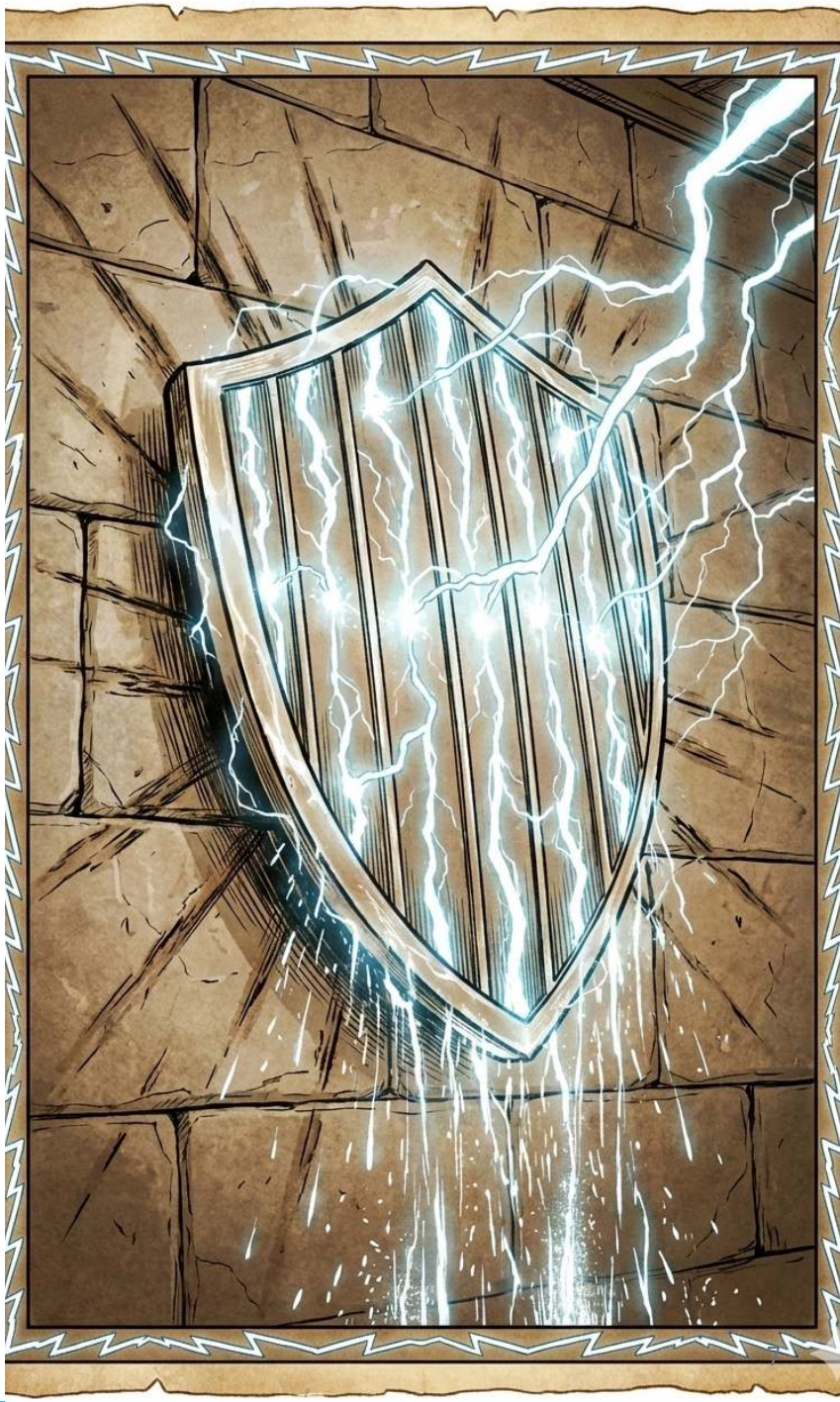


Most of the code is generated, but you can't prompt confidence.

When Code Generation Scales …

… **verification** becomes the bottleneck.

So we need a **safety net**: a test suite we can trust.



What We Expect From the Safety Net

Fast

Feedback in **minutes**, not hours

so we run it on every change

Comprehensive

Enough **correct tests**

for confidence in every commit

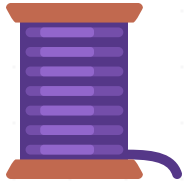
Today we focus on fast tests.

How Do You Get Fast Tests?



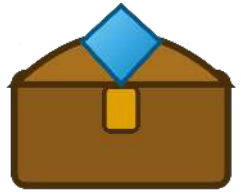
No tests at all

fastest build, zero confidence



Parallelize tests

use the cores you pay for

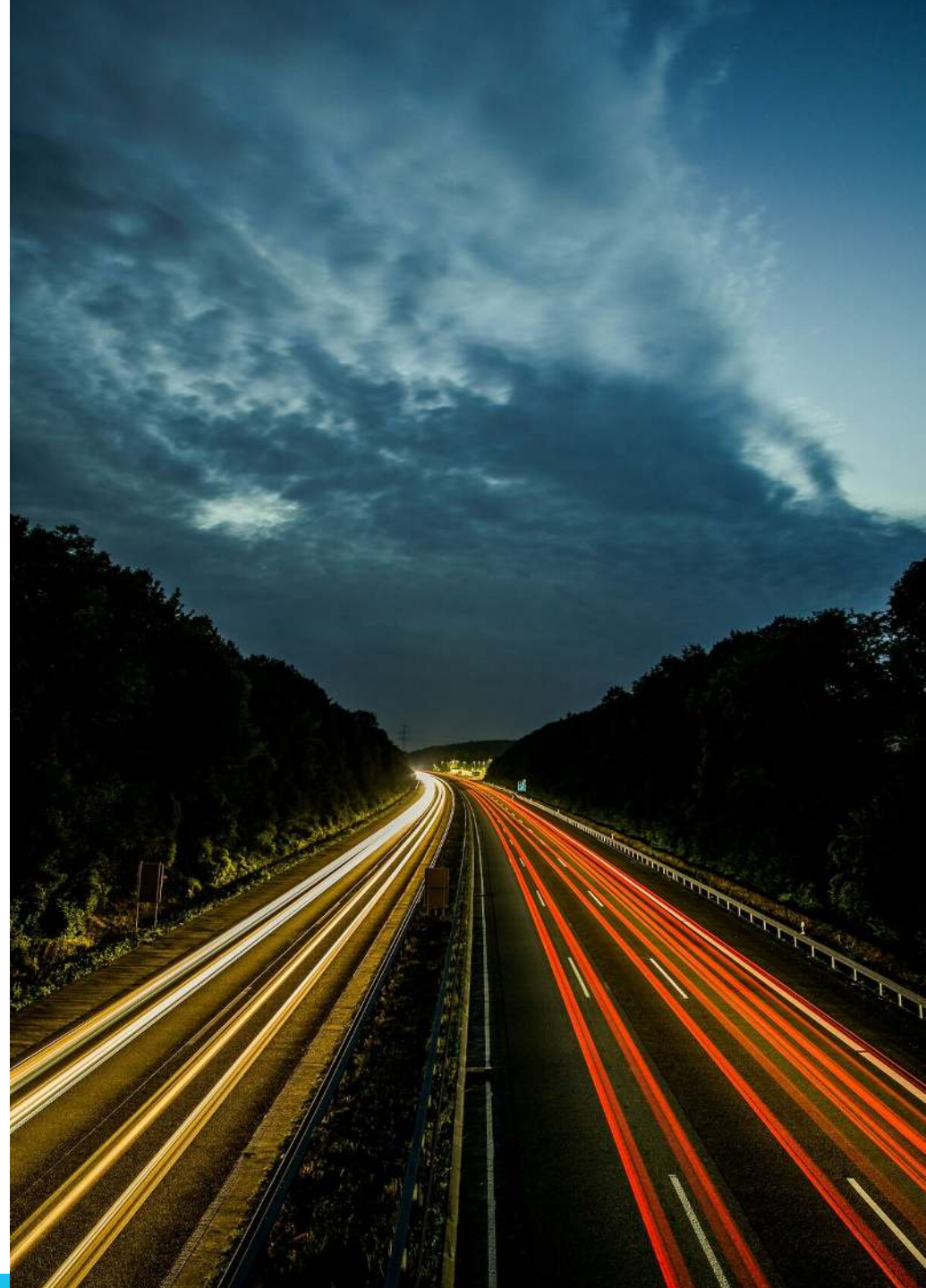


Something else ...

a treasure that helps us

THE SPRING TEST HIDDEN GEM

Reusing an ApplicationContext for tests



Philip Riecks · @rieckpil · PragmaTech GmbH

The Culprit: Every Test Starts a Context

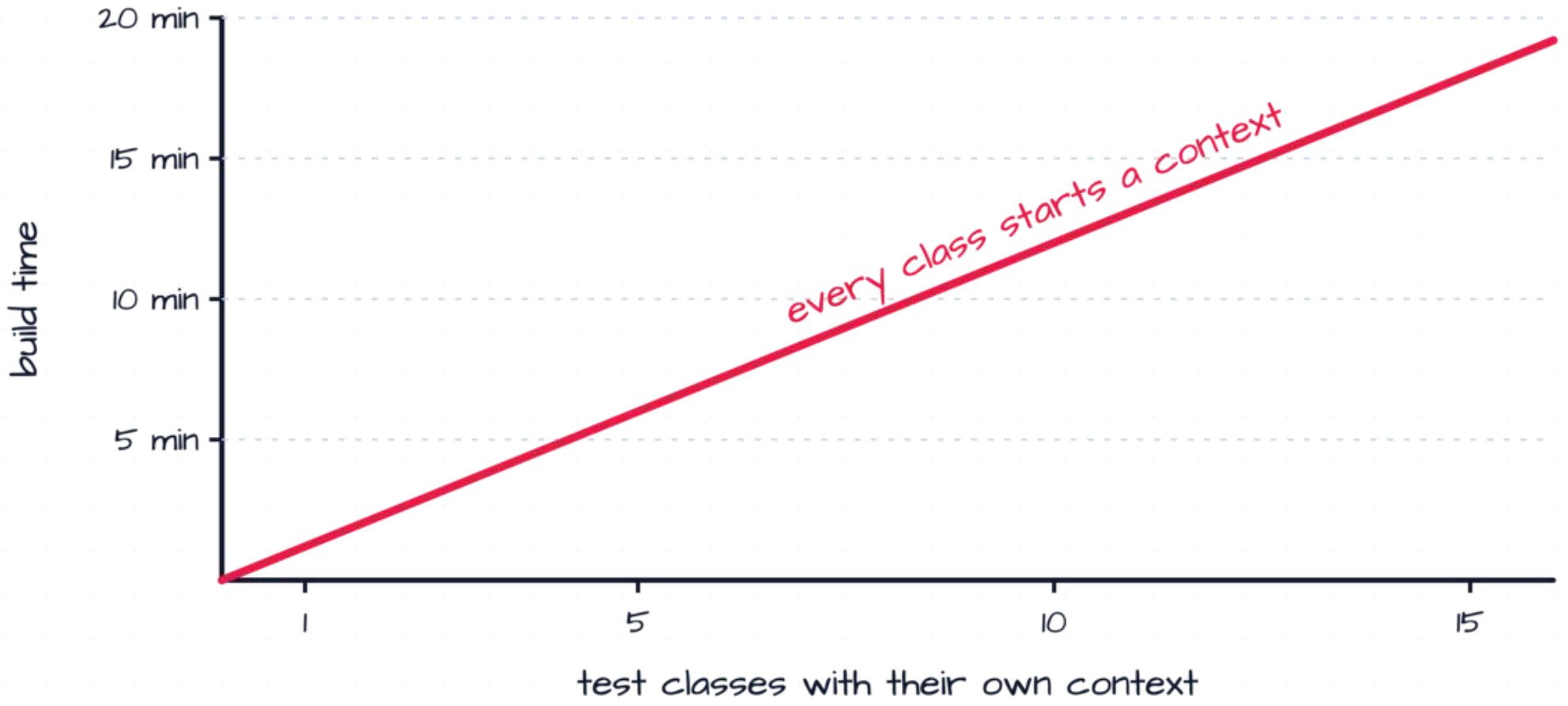
Whether sliced or full, a Spring test needs an `ApplicationContext`:

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.1.1)

10:19:16.157 INFO  BookApiSpringBootTest : Starting BookApiSpringBootTest using Java 25.0.2
10:19:16.548 INFO  RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories
10:19:16.863 INFO  TomcatWebServer : Tomcat initialized with port 0 (http)
10:19:17.064 INFO  org.hibernate.orm.core : HHH000001: Hibernate ORM core version 7.4.5.Final
...
10:19:17.824 INFO  LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory
...
10:19:18.320 INFO  BookApiSpringBootTest : Started BookApiSpringBootTest in 2.289 seconds
```

To Avoid: Linear Build Time Growth



Lucky for us, Spring Test has a fix built in.

Spring TestContext Context Caching

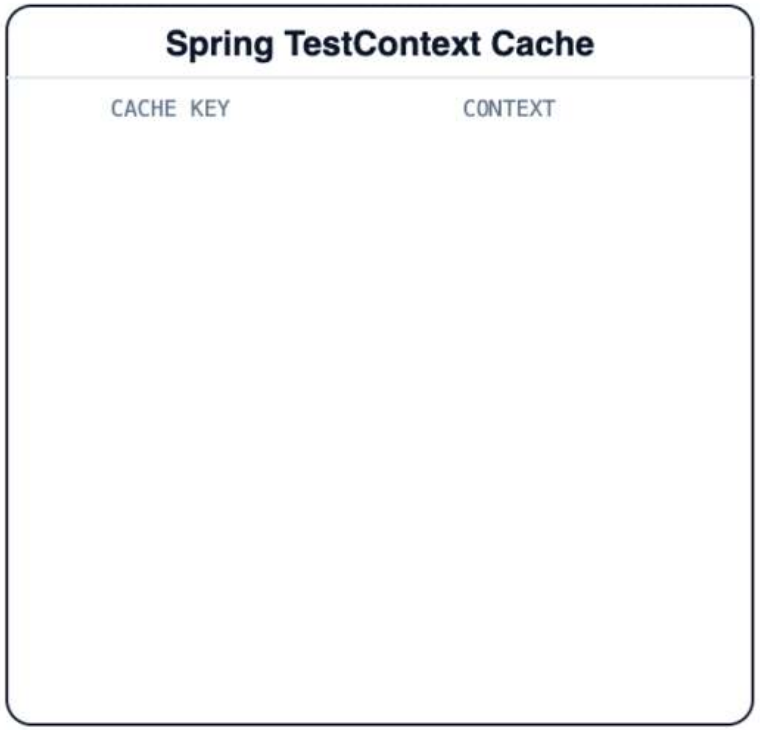
- The **TestContext framework** caches an already started `ApplicationContext`
- A test with the **same configuration** gets the cached context instantly
- Available **out of the box** - nobody has to switch it on
- Part of Spring Framework's test support, so it works for JUnit 4, 5 and every slice

Context Caching in Action

OrderIT
unique context configuration
#327321289

PaymentIT
unique context configuration
#327321289

CheckoutIT
unique context configuration
#565212314



How the Cache Key Is Built

```
// DefaultContextCache.java
private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
  Collections.synchronizedMap(new LruCache(32, 0.75f));
```

This goes into the cache key (`MergedContextConfiguration`):

- activeProfiles (`@ActiveProfiles`)
- propertySourceProperties (`@TestPropertySource`, `properties = ...`)
- contextCustomizers (`@MockitoBean`, `@MockitoSpyBean`, `@DynamicPropertySource`, ...)
- configuration classes, initializers, web environment, etc.

Every unique combination is a new context start.

What Is ”Bad” for Context Caching Here?

```java
@DirtiesContext
@Testcontainers
@ActiveProfiles("integration-test")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
abstract class AbstractIntegrationTest {
```

What's "bad" for context caching here?

```java
@ActiveProfiles("integration-test")
@Import(SomeTestConfiguration.class)
@ContextConfiguration(initializers = CustomInitializer.class)
@SpringBootTest(properties = {"features.login-enabled=true", "custom.message=duke42"})
class ShowcaseIT {

  @MockBean
  private OrderService orderService;

  @SpyBean
  private CustomerService customerService;

  @Test
  void shouldInitializeContext(@Autowired ApplicationContext applicationContext) {
    assertThat(applicationContext)
            .isNotNull();
  }
}
```

Hit or Miss?

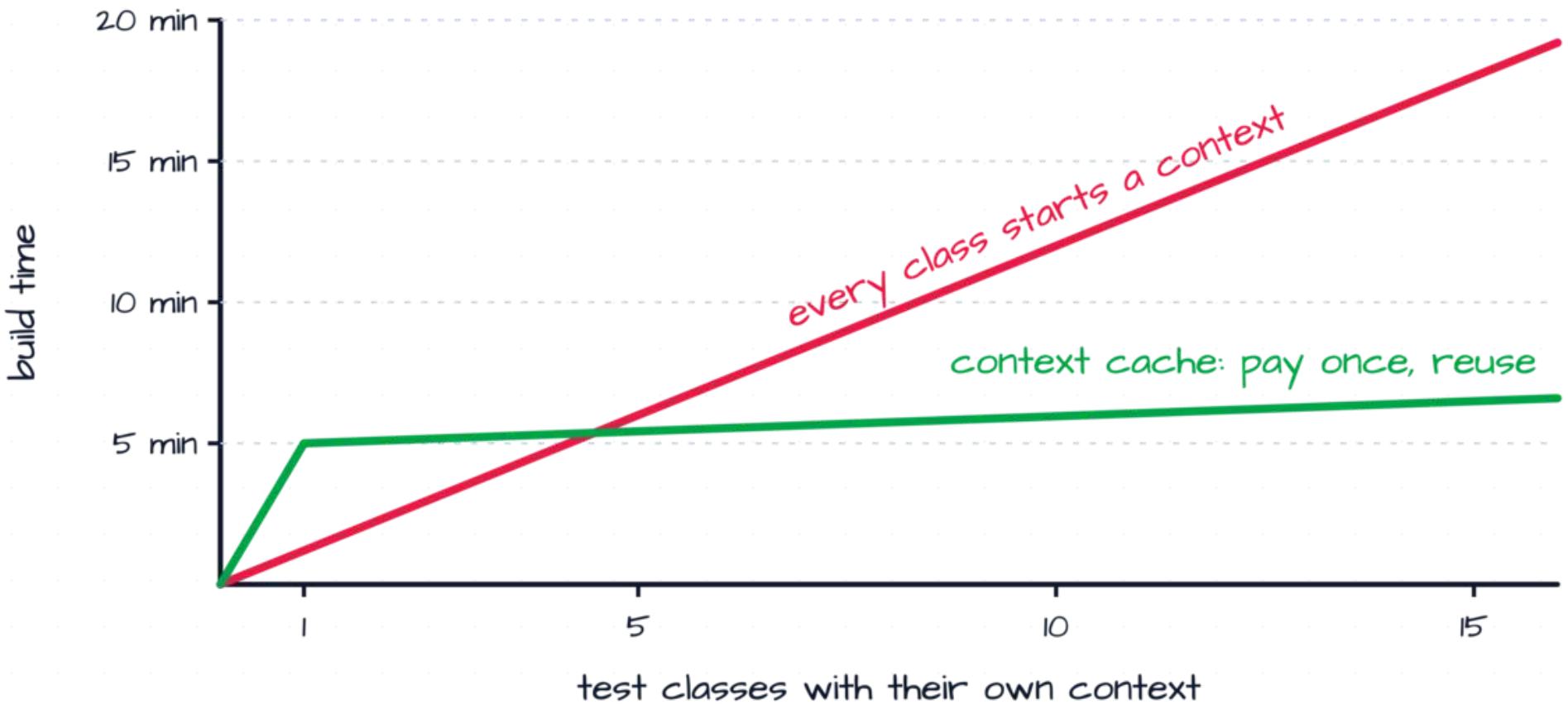
Cache hit: same key

- Same annotations on the test class
- Same properties and profiles
- Same mocks and spies
- Context is reused, cost close to zero

Cache miss: one tiny difference

- Extra `properties = ...` on one class
- A `@MockitoBean` only in one test
- `@DirtiesContext` closes the context
- New context start, pay full price

The Gem in One Picture



You pay for a context once per unique configuration, not once per test class.

DEMO

A small Spring Boot CRUD API and its test suite

Spring Boot 4.1, Java 25, 5 test classes, 9 tests.



The Demo Application

- Book CRUD API: `GET/POST/PUT/DELETE /api/books`
- Spring Web MVC, Spring Data JPA, H2
- 5 test classes, all `@SpringBootTest`, each a little different

Test class	What is different
`BookApiSpringBootTest`	`RANDOM_PORT` and `TestRestTemplate`
`BookControllerSpringBootTest`	`@MockitoBean` and `MockMvc`
`BookServiceSpringBootTest`	`@MockitoSpyBean`
`BookDeleteSpringBootTest`	`@DirtiesContext`
`BookRepositorySpringBootTest`	one extra `property`

Three Steps to Faster Builds

01 Understand - measure the context starts with the Spring Test Profiler

02 Optimize - align the configurations so contexts get reused

03 Ensure - keep it fast, for your team and for your AI agents

STEP 1

Understand: measure before you guess

Add the Spring Test Profiler

An open-source Spring Test utility that visualizes context caching for your test suite:

```
<dependency>
  <groupId>digital.pragmatech.testing</groupId>
  <artifactId>spring-test-profiler</artifactId>
  <version>0.3.1</version>
  <scope>test</scope>
</dependency>
```

Activate It

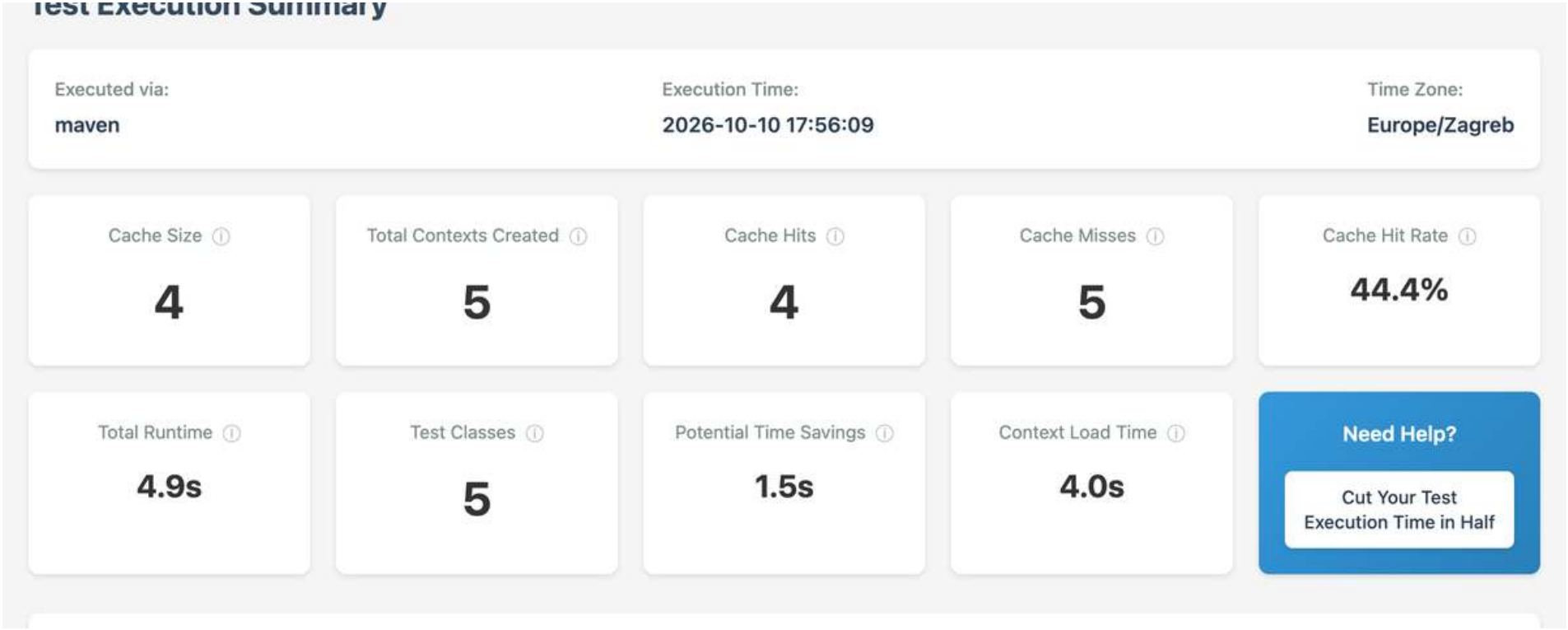
`src/test/resources/META-INF/spring.factories`:

```
org.springframework.test.context.TestExecutionListener=\
digital.pragmatech.testing.SpringTestProfilerListener
org.springframework.context.ApplicationContextInitializer=\
digital.pragmatech.testing.diagnostic.ContextDiagnosticApplicationInitializer
```

Run the tests, open the report:

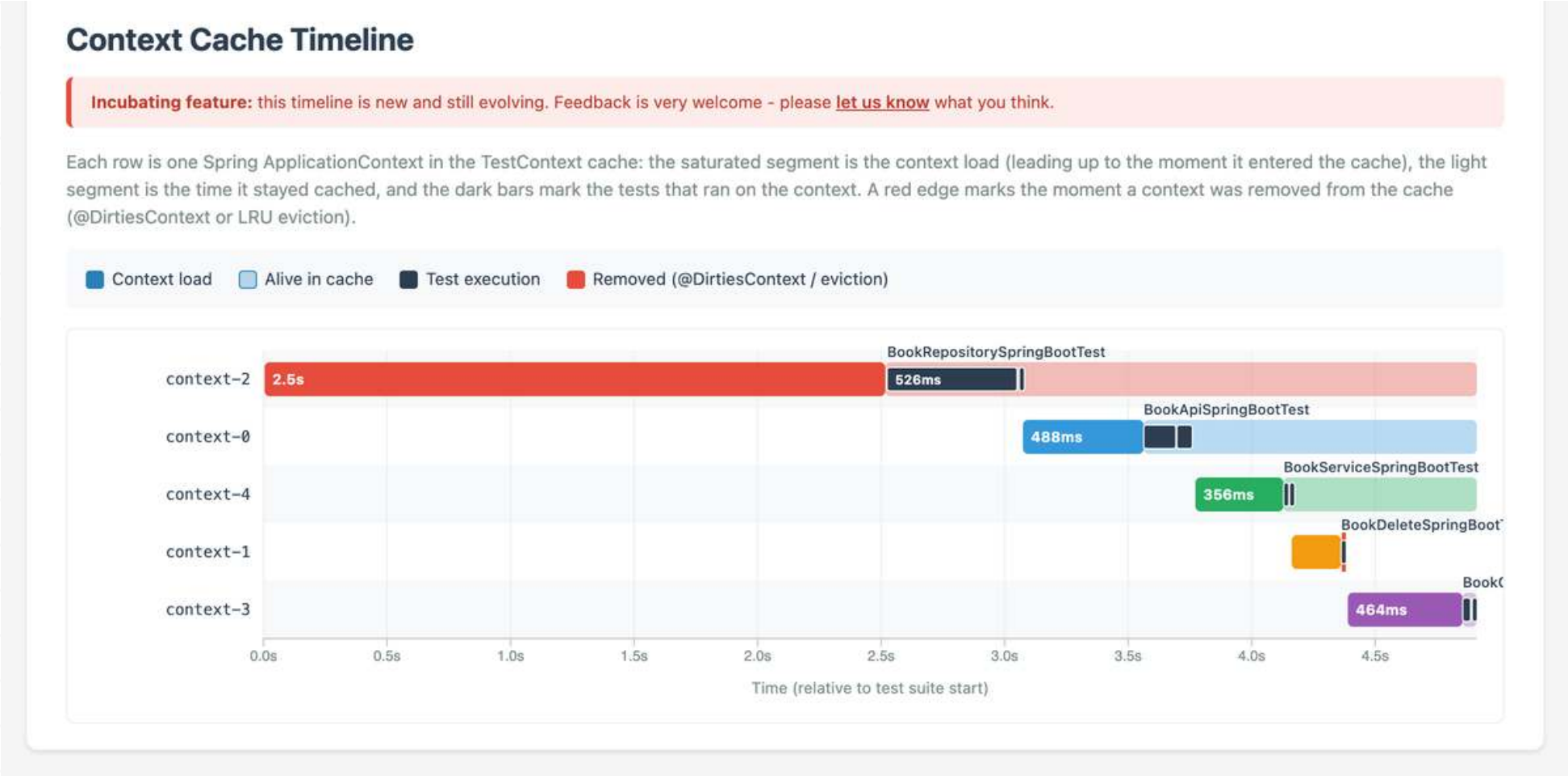
```
./mvnw verify
open target/spring-test-profiler/latest.html
```

The Baseline: 5 Test Classes, 5 Contexts

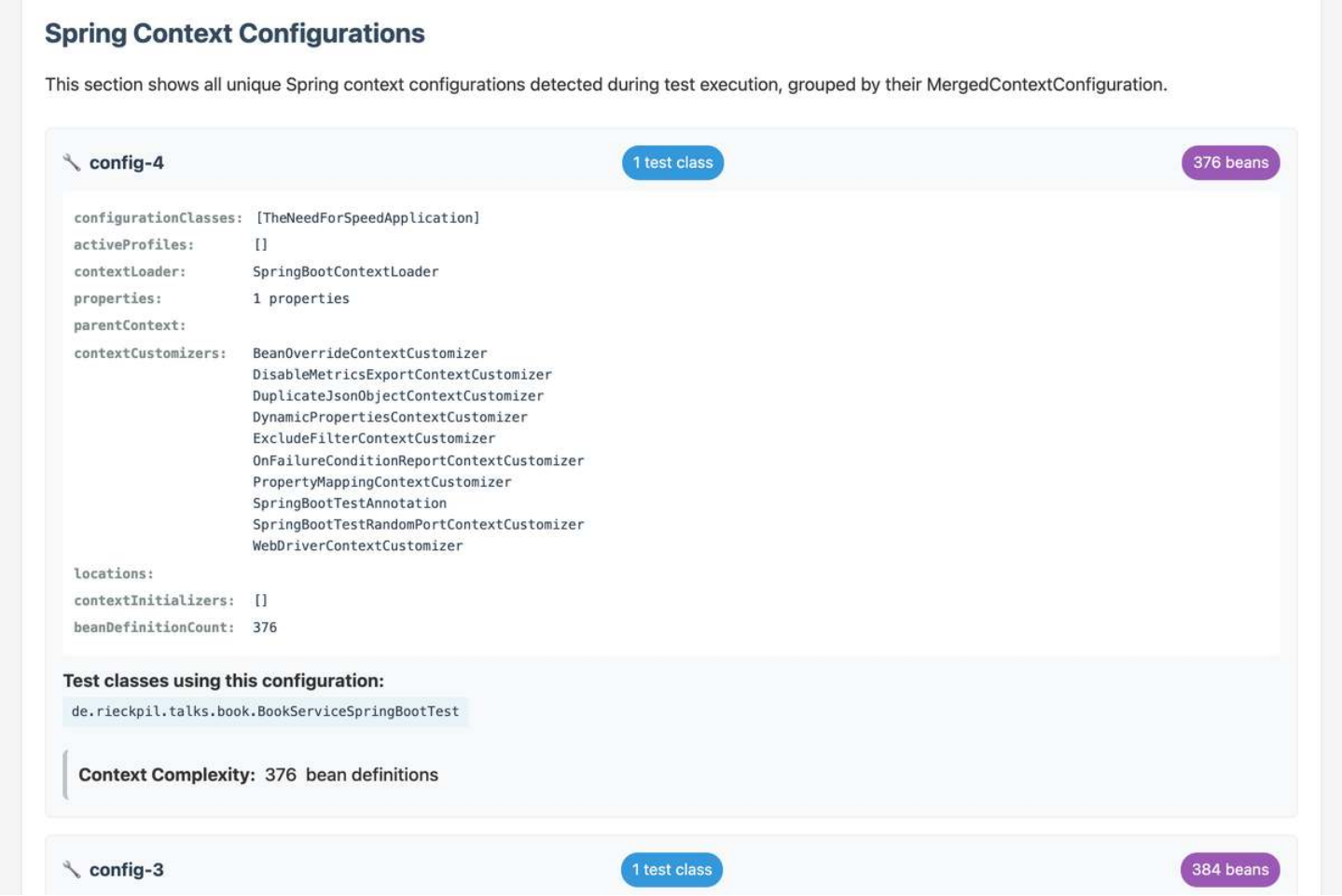


Cache hit rate 44.4%, **4.0s** of the 4.9s are context startup.

The Timeline: Five Contexts, One per Class



Why Do the Contexts Differ?



Spring Context Configurations

This section shows all unique Spring context configurations detected during test execution, grouped by their MergedContextConfiguration.

config-4 1 test class 376 beans

```
configurationClasses: [TheNeedForSpeedApplication]
activeProfiles:       []
contextLoader:        SpringBootContextLoader
properties:           1 properties
parentContext:
contextCustomizers:   BeanOverrideContextCustomizer
                      DisableMetricsExportContextCustomizer
                      DuplicateJsonObjectContextCustomizer
                      DynamicPropertiesContextCustomizer
                      ExcludeFilterContextCustomizer
                      OnFailureConditionReportContextCustomizer
                      PropertyMappingContextCustomizer
                      SpringBootTestAnnotation
                      SpringBootTestRandomPortContextCustomizer
                      WebDriverContextCustomizer
locations:
contextInitializers:  []
beanDefinitionCount:  376
```

Test classes using this configuration:

`de.rieckpil.talks.book.BookServiceSpringBootTest`

> **Context Complexity:** 376 bean definitions

config-3 1 test class 384 beans

STEP 2

Optimize: from five contexts to one



Three Levers to Reuse Contexts

1. **Abstract integration test class** - one place for all context configuration
2. **Context initializer /** `@DynamicPropertySource` - shared setup, same key for everyone
3. **Fewer mocks and no** `@DirtiesContext` - use real beans and clean up your own data

Align first, then slice only where it really pays off.

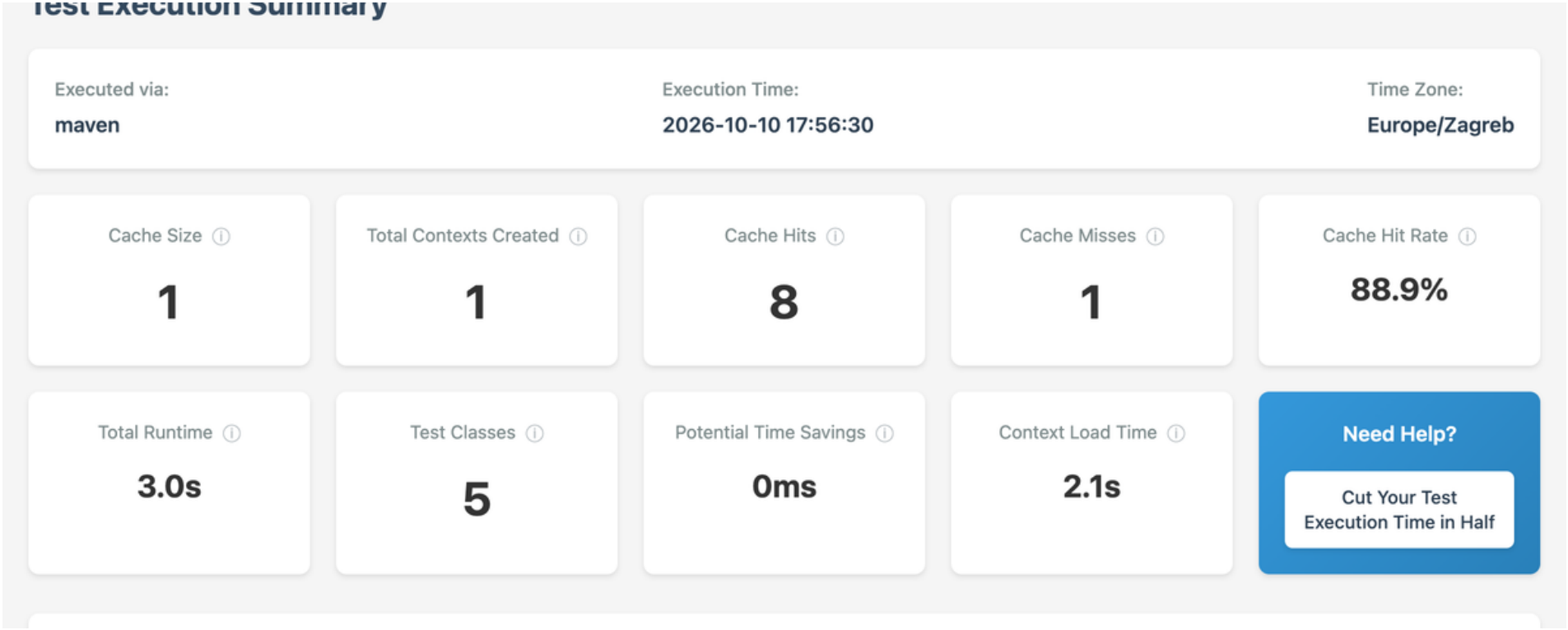
The Fix: One Base Class

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@AutoConfigureMockMvc
@AutoConfigureTestRestTemplate
abstract class AbstractIntegrationTest {
}

class BookApiSpringBootTest
    extends AbstractIntegrationTest {
  // no own annotations anymore
}
```

- One annotation set for all tests
- No `@MockitoBean` or `@MockitoSpyBean`
- No `@DirtiesContext`
- No extra `properties`

The Result: 1 Context



Cache hit rate 88.9%, 1 context created, total runtime 4.9s down to 3.0s.

Demo Result

5 → 1

Contexts created

44% → 89%

Cache hit rate

-39%

Total runtime on a tiny suite

STEP 3

Ensure: keep it fast



Educate Your Team

- Explain the **cache key**: every new annotation combination costs a context start
- Put shared configuration into **one base class**, and review changes to it
- Treat `@DirtiesContext` and a new `@MockitoBean` like a performance decision
- Run the profiler report from time to time, share it in the team

Educate Your AI Agents Too

Agents write most of the tests now. They need the same rules:

```
.claude/skills/
├── unit-testing              fast tests without context bloat
├── slice-testing             right-sized Spring context slices
├── integration-testing       extend AbstractIntegrationTest, no new context
└── test-setup-reviewer       flags context cache killers
```

Point the skill to your `AbstractIntegrationTest` and name the antipatterns.

Let the Pipeline Guard the Context Count

The profiler writes `target/spring-test-profiler/results.json`:

```
./mvnw verify

expectedContexts=1
actualContexts=$(jq -r '.contextsCreated' \
  target/spring-test-profiler/results.json)

if [ "$actualContexts" -gt "$expectedContexts" ]; then
  echo "Too many Spring contexts: $actualContexts"
  exit 1
fi
```

A new `@DirtiesContext` or a stray `@MockitoBean` now fails the build.

Summary

01 Every context start costs time, and the build time grows with every test class

02 Context caching is the hidden gem: same configuration, same context

03 Understand with the profiler, optimize with a base class, ensure with CI and skills

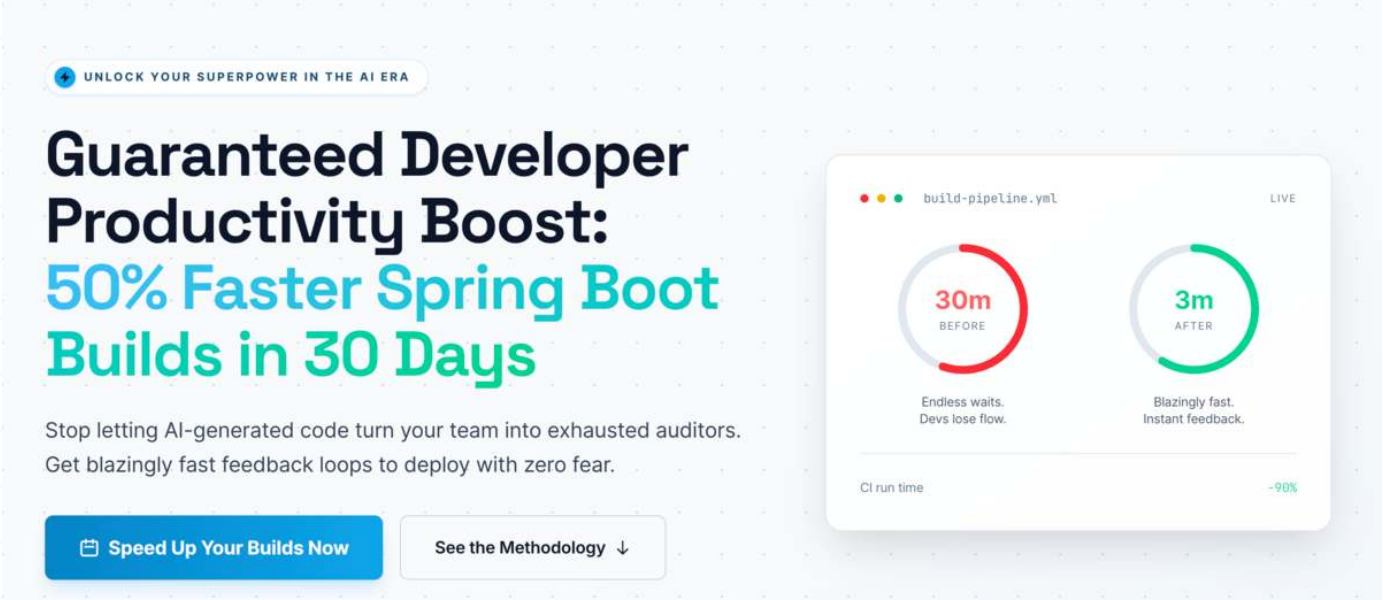
Fast feedback is a feature.

The treasure was in your project all along.

Go and find it.



Need Help Optimizing Your Build?



- Spring Boot test suite too slow? Let's find your context cache killers together
- Test audits, team workshops and hands-on pairing at pragmatech.digital
- **Reach out:** philip@pragmatech.digital or LinkedIn

Joyful (and fast) testing!

The **Spring Boot Testing Newsletter**: best practices, recipes and quick wins in your inbox - **rieckpil.de/newsletter**

